

Legal Query System

Interview Preparation Guide

Complete Workflow, Techniques & Cross Questions

April 2026

Contents

1. Project Overview & One-Liner
2. End-to-End Workflow
3. Step 1: PDF Text Extraction — PyPDF2
4. Step 2: Text Cleaning — Regular Expressions
5. Step 3: Metadata Detection — Filename Parsing
6. Step 4: Legal Chunking — Section-Aware Splitting
7. Step 5: Embedding Generation — Sentence Transformers
8. Step 6: Vector Store — ChromaDB
9. Step 7: Semantic Retrieval — Cosine Similarity
10. Step 8: Context Formatting
11. Step 9: LLM Generation — Gemini 2.0 Flash (Vertex AI)
12. Step 10: Source Filtering & Response Construction
13. System Architecture Overview
14. Interview Cross Questions & Answers
15. Quick Revision Cheat Sheet

1. Project Overview & One-Liner

Elevator Pitch (use this in interviews):

"I built a Legal Query System — an AI-powered chatbot that lets users ask legal questions in plain English and get accurate, cited answers from 18 Indian laws. It uses RAG (Retrieval-Augmented Generation) to first search a vector database of legal documents and then feed the relevant sections to Google Gemini to generate responses with exact Act names and Section numbers."

What the project does

It is a full-stack web application where users sign up, log in, and ask legal questions through a chat interface. Behind the scenes, the system searches through 18 major Indian legal documents (stored as vector embeddings), retrieves the most relevant sections, feeds them to a large language model, and returns an answer with proper legal citations.

Why this project matters

- Solves a real problem — legal information is inaccessible to common people.
- Uses cutting-edge AI (RAG + LLM) — not just a basic chatbot.
- Demonstrates full-stack skills — React frontend, Node.js/Express backend, Python ML pipeline.
- Shows understanding of NLP concepts — embeddings, vector search, prompt engineering.
- Production-ready features — JWT auth, session management, multi-turn chat, source filtering.

2. End-to-End Workflow

The project has two main phases:

Phase A: Offline — PDF Ingestion (runs once)

This phase processes all legal PDFs into a searchable vector database:

- Step 1: Extract raw text from each PDF using PyPDF2.
- Step 2: Clean the extracted text (remove headers, footers, gazette noise).
- Step 3: Detect metadata (Act name, year) from the PDF filename.
- Step 4: Chunk the text by legal sections (section-aware splitting).
- Step 5: Convert each chunk into a 384-dimensional vector embedding using Sentence Transformers.
- Step 6: Store all embeddings + metadata in ChromaDB (persistent vector database).

Phase B: Online — Query Processing (runs per query)

This phase handles every user question in real time:

- Step 7: Convert the user's question into a vector and find the top-6 most similar chunks (semantic retrieval).
- Step 8: Format the retrieved chunks into a structured context string with headers.
- Step 9: Send the context + question + system prompt to Gemini 2.0 Flash LLM for answer generation.
- Step 10: Filter out low-relevance sources (< 45% similarity) and construct the final response with citations.

Key insight for interviews: The offline phase is $O(n)$ — it processes all documents once. The online phase is $O(\log n)$ per query thanks to the HNSW index in ChromaDB, making retrieval sub-linear and fast even with thousands of chunks.

3. Step 1: PDF Text Extraction

Tool Used: PyPDF2

What it is: A pure-Python library for reading PDF files. It can extract text, metadata, and manipulate PDF pages.

Why we chose it: Lightweight, no system dependencies (unlike pdfminer or Tesseract), works well for text-based PDFs (our legal docs are digitally created, not scanned).

How it works in our project

- Opens each PDF file in binary read mode.
- Iterates through every page of the PDF using PdfReader.
- Calls `extract_text()` on each page to get the text content.
- Concatenates all page texts with page markers (e.g., "----- PAGE 1 -----") for traceability.
- Saves the result as a `.txt` file in the `extracted_text/` directory.

Limitations to mention in interviews

- Cannot extract text from scanned PDFs (would need OCR like Tesseract).
- Sometimes loses formatting — columns, tables, and special characters may not extract perfectly.
- Does not preserve the visual layout of the PDF.

4. Step 2: Text Cleaning

Tool Used: Regular Expressions (Python `re` module)

What it is: Regex is a pattern-matching language built into Python. It lets you find and replace complex text patterns.

Why we need it: PDF-extracted text is noisy — it contains gazette headers, page markers, extra whitespace, and formatting artifacts that would confuse the AI model.

What we clean

- Page markers like "----- PAGE 33 -----" are removed.
- Gazette headers like "THE GAZETTE OF INDIA EXTRAORDINARY" are stripped.
- Multiple spaces/tabs are collapsed into a single space.

- Excessive newlines (3+) are collapsed into double newlines (paragraph breaks).

Interview point: This is a classic NLP preprocessing step. Cleaning ensures that the embedding model focuses on actual legal content rather than noise, improving retrieval accuracy.

5. Step 3: Metadata Detection

Technique: Filename Parsing

How it works: The PDF filename itself encodes the Act name and year. For example, "indian_contract_act_1872.pdf" tells us the document is the Indian Contract Act from 1872.

Parsing algorithm

- Check if the filename matches a special-case dictionary (for names like "engaadhaar" that don't follow conventions).
- Extract a trailing 4-digit year using regex (e.g., 1872, 2023).
- Remove the year portion from the filename string.
- Replace underscores with spaces and apply title-casing.
- Result: ("Indian Contract Act, 1872", "1872").

Why not extract metadata from the PDF content?

Legal PDFs have inconsistent internal formatting — some put the Act name on page 1, others have it buried in headers. Filename-based detection is simple, reliable, and 100% accurate for our controlled dataset. If we were accepting user-uploaded PDFs, we would need content-based extraction.

6. Step 4: Legal Chunking

Technique: Section-Aware Splitting

What chunking means: Breaking a large document into smaller pieces (chunks) that can each be individually embedded and retrieved.

Why not fixed-size chunking? Legal text has a natural structure — Sections, Chapters, Clauses. If you split at every 500 characters, you will cut a section in half, losing its legal meaning. Our approach respects the document structure.

How our chunking works (step by step)

- CLEAN: Remove PDF noise from the raw text.
- DETECT SECTIONS: Use regex to find section boundaries (pattern: number + period + title, e.g., "73. Compensation for loss").
- TRACK CHAPTERS: Continuously watch for chapter headers ("CHAPTER IV — ...") and update the

current chapter label.

- CAPTURE PREAMBLE: Any text before the first section is saved as a "Preamble" chunk.
- SPLIT LARGE SECTIONS: If a section is > 3,000 characters, split it at sentence boundaries (period + space) within a 1,500-character window.
- SKIP SHORT FRAGMENTS: Sections < 70 characters (likely ToC entries) are discarded.
- STORE: Each chunk is saved as a JSON record with text + metadata (act_name, year, chapter, section, section_title).

Chunk metadata structure

Each chunk contains: act_name (e.g., "Indian Contract Act, 1872"), year ("1872"), source_file, chapter ("CHAPTER VI — Consequences of Breach"), section ("73"), section_title ("Compensation for loss or damage"). This metadata is what enables precise legal citations in the final answer.

7. Step 5: Embedding Generation

Model Used: Sentence Transformers (all-MiniLM-L6-v2)

What it is: A pre-trained transformer model from Hugging Face that converts any text into a fixed-size vector (384 dimensions). It is fine-tuned specifically for semantic similarity tasks.

Architecture: It is based on Microsoft's MiniLM, which is a distilled (compressed) version of BERT. It has 6 layers (L6), 384 hidden dimensions, and 22.7 million parameters.

Why this model? It is small (80 MB), fast (can embed 1000+ sentences/second on CPU), and performs well on semantic similarity benchmarks. Perfect for our use case where we need speed without a GPU.

How embeddings work (explain this in interviews)

An embedding model takes a text string (like "What is the punishment for theft?") and converts it into a list of 384 numbers (called a vector). The key property is that texts with similar meanings produce vectors that are close together in 384-dimensional space. So "punishment for theft" and "penalty for stealing" would have vectors pointing in nearly the same direction, even though they use different words.

This is fundamentally different from keyword search. Keyword search would fail if the user says "stealing" but the document says "theft". Embedding-based search understands that these mean the same thing.

Technical details for interviews

- Input: Any text string (our chunks are typically 200-1500 characters).
- Output: A 384-dimensional float vector (numpy array).
- Training: Pre-trained on 1 billion+ sentence pairs from NLI and paraphrase datasets.
- Tokenizer: WordPiece tokenizer (same as BERT), max 256 tokens input.
- Batch processing: We embed in batches of 32 for efficiency.
- The model is loaded once (singleton pattern) and reused for all queries.

Common interview question: "Why not use OpenAI embeddings or a larger model?"

Answer: all-MiniLM-L6-v2 runs entirely locally (no API costs), is fast on CPU, and is sufficient for our domain. Larger models would give marginal improvement but add latency and cost. For a production system with millions of documents, we might consider fine-tuning or using a legal-specific embedding model.

8. Step 6: Vector Store — ChromaDB

What is ChromaDB?

ChromaDB is an open-source, lightweight vector database designed for AI applications. It stores text documents along with their vector embeddings and metadata, and allows fast similarity searches.

Why ChromaDB (vs. Pinecone, Weaviate, FAISS)?

- Runs locally — no cloud setup or API keys needed (unlike Pinecone).
- Persistent storage — data survives restarts (stored on disk).
- Built-in metadata filtering — can filter by `act_name`, `year`, etc.
- Simple Python API — easy to integrate with our Flask backend.
- Supports cosine similarity natively via HNSW (Hierarchical Navigable Small World) index.

How we use it

- Create a collection called "legal_docs" with cosine distance metric.
- Store each chunk with: its text (document), its embedding (384-dim vector), its metadata (`act_name`, `section`, etc.), and a unique ID.
- At query time, pass the query embedding and retrieve top-k most similar documents.
- HNSW index enables approximate nearest neighbor search in $O(\log n)$ time.

HNSW Index (explain if asked)

HNSW (Hierarchical Navigable Small World) is the indexing algorithm ChromaDB uses. It builds a multi-layer graph where each node is a vector. Higher layers have fewer, more spread-out nodes (for coarse search), and lower layers have more nodes (for fine-grained search). During a query, the algorithm starts at the top layer and navigates down, progressively narrowing the search. This makes it much faster than a brute-force comparison against every vector.

9. Step 7: Semantic Retrieval

What happens at query time

- User types a question, e.g., "What are my rights if I am arrested?"
- The question is converted to a 384-dim vector using the same all-MiniLM-L6-v2 model.
- ChromaDB searches its HNSW index and returns the top-6 chunks with the highest cosine similarity.
- Each result includes: the chunk text, its metadata, and the cosine distance.

Cosine Similarity (explain this)

Cosine similarity measures the angle between two vectors. If two vectors point in the same direction, cosine similarity = 1 (perfectly similar). If perpendicular, similarity = 0 (unrelated). If opposite, similarity = -1 (opposite meaning). We convert ChromaDB's cosine distance to similarity using: similarity = 1 - distance.

Why top-6?

We retrieve 6 chunks to give the LLM enough context. A legal question might relate to multiple sections or even multiple Acts. Too few chunks (1-2) might miss important context. Too many (10-20) would add noise and exceed the LLM's context window. 6 is a good balance based on our testing.

10. Step 8: Context Formatting

Why format the context?

The LLM needs to know WHERE each piece of text comes from in order to generate proper citations. Raw text without labels would make it impossible for the model to say "According to Section 73 of the Indian Contract Act..."

How we format

Each retrieved chunk is prefixed with a structured header:

[Source: Indian Contract Act, 1872, Section 73 — Compensation for loss | CHAPTER VI | Relevance: 89.34%]

...followed by the actual chunk text. All chunks are separated by "---" dividers.

This format is specifically designed for the LLM — the square brackets and labels make it easy for the model to parse and reference the source information in its answer.

11. Step 9: LLM Generation

Model Used: Google Gemini 2.0 Flash

What it is: Gemini 2.0 Flash is a large language model from Google, accessed through the Vertex AI cloud platform. It is optimized for speed ("Flash") while maintaining high quality.

Access method: We use the LangChain library (`langchain_google_vertexai`) to connect to the Gemini API using a GCP service account credential file.

Temperature: Set to 0.2 (low). This makes responses more deterministic and focused — critical for legal accuracy where we don't want creative or variable outputs.

Max tokens: 2048 tokens, which is enough for a detailed legal explanation with citations.

System Prompt (Prompt Engineering)

The system prompt is the most critical piece of the pipeline. It instructs the LLM on exactly how to behave:

- For legal questions: Start with a simple explanation, then give the formal legal definition, then cite the exact Act, Section, and clause text.
- For casual talk (hello, what can you do?): Respond naturally like a human, no legal jargon.
- Never hallucinate: If the context doesn't contain the answer, say "I don't have specific information about this."
- Never reveal implementation: Don't discuss AI architecture, training, or internal workings.
- Always add a disclaimer: "This information is for general awareness only. Please consult a qualified legal professional."
- The prompt also lists all 18 legal topics the system covers, so it can describe its capabilities naturally.

Multi-Turn Conversation

The system supports follow-up questions by passing the last 6 messages (3 exchanges) as conversation history. The LangChain library converts these into `HumanMessage` and `AIMessage` objects that Gemini understands. This lets users say things like "Tell me more about that section" without repeating context.

Interview point: "Why Gemini and not GPT-4 or Claude?"

Answer: Gemini 2.0 Flash offers a good balance of quality, speed, and cost. It integrates natively with Google Cloud (Vertex AI), where we already host our credentials. The Flash variant is optimized for low latency while maintaining strong reasoning. However, the architecture is LLM-agnostic — swapping to GPT-4 or Claude would only require changing the LangChain model class.

12. Step 10: Source Filtering & Response Construction

Source Filtering

Threshold: 0.45 (45% cosine similarity). Any source below this is not shown to the user.

This solves a subtle but important problem: when a user says "Hello, how are you?", the retrieval still returns 6 chunks (because it always returns top-k), but all of them have very low similarity scores (0.1-0.3). Without filtering, we would show irrelevant legal citations alongside a casual greeting. The 0.45 threshold ensures only genuinely relevant sources appear.

Source Text Cleaning

- Leading section numbers (like "73. ") are removed from the display text since the section number is already shown in metadata.
- Multiple whitespace is collapsed into single spaces.
- Long texts are truncated at sentence boundaries (at ~1200 characters) to keep the UI clean.

Final Response Structure

The API returns a JSON object with two fields:

- "answer": The complete natural-language response from the LLM with citations woven into the text.
- "sources": An array of source objects, each containing `act_name`, `section`, `section_title`, `chapter`, `year`, `text excerpt`, and `similarity score`.

13. System Architecture Overview

Three-Tier Architecture

- Frontend (React + Vite): 4 pages — Login, Signup, Dashboard, Chat. Uses Axios for API calls with JWT token in headers.
- Application Server (Node.js + Express on port 3001): Handles auth (JWT + bcrypt), chat session CRUD, and proxies queries to the RAG API. Connects to MongoDB.
- RAG API Server (Python + Flask on port 5000): Hosts the ML pipeline — embedding model, ChromaDB, Gemini LLM. Exposes `/api/query` and `/api/ingest` endpoints.

Request Flow (for a query)

- 1. User types question in React chat UI.
- 2. Frontend sends POST `/api/chat/sessions/{id}/query` with JWT token in header.
- 3. Express server verifies JWT, loads the session from MongoDB, builds chat history.
- 4. Express server forwards query + chat_history to Flask RAG API (POST `/api/query`).
- 5. Flask server runs RAG pipeline: embed query → retrieve → format → generate → filter.
- 6. Flask returns `{answer, sources}` to Express server.
- 7. Express server saves user message + assistant response in MongoDB session.
- 8. Express server returns the response to the React frontend.
- 9. React renders the answer with expandable source citations.

Database Schema (MongoDB)

- User collection: name, email, password (bcrypt hash), timestamps.
- ChatSession collection: userId (ref), title, messages[] (role, content, sources[], timestamp), isSolved flag, timestamps.

14. Interview Cross Questions & Answers

These are the most likely questions an interviewer will ask about this project. Practice answering them out loud.

A. About RAG & Architecture

Q1. What is RAG and why did you use it instead of fine-tuning?

Ans: RAG stands for Retrieval-Augmented Generation. Instead of fine-tuning an LLM on legal data (which is expensive, requires large datasets, and makes the model outdated when laws change), RAG retrieves relevant documents at query time and feeds them as context to the LLM. This means: (1) the knowledge base can be updated just by adding new PDFs — no retraining needed, (2) every answer is traceable to a source document, (3) it dramatically reduces hallucination because the model works from provided text, not its own potentially outdated training data.

Q2. What happens if the user asks a question your knowledge base doesn't cover?

Ans: The retrieval step will still return 6 chunks, but they will all have low similarity scores. Two things protect us: (1) the source filter (threshold 0.45) will remove all weak matches so no misleading citations are shown, (2) the system prompt explicitly instructs the LLM to say "I don't have specific information about this" when the context doesn't contain the answer.

Q3. Why two separate backend servers? Why not just one?

Ans: Separation of concerns. The Python/Flask server handles the ML-heavy work (loading models, running embeddings, calling Gemini). The Node.js/Express server handles web application logic (authentication, session management, database operations). This means we can scale them independently — for example, running multiple Flask instances behind a load balancer for ML processing while keeping a single Express server for auth. It also means a crash in the ML pipeline doesn't bring down the auth system.

Q4. How would you scale this system for millions of users?

Ans: Several approaches: (1) Use a managed vector database like Pinecone or Weaviate instead of local ChromaDB for distributed search. (2) Cache frequent queries using Redis. (3) Run multiple Flask RAG workers behind Nginx/load balancer. (4) Use async processing — queue queries and return results via WebSocket. (5) Move to a faster embedding model or pre-compute embeddings for common queries.

B. About Embeddings & Vector Search

Q5. What is the difference between keyword search and semantic search?

Ans: Keyword search (like SQL LIKE or Elasticsearch) matches exact words. If the user says "stealing" but the document says "theft", keyword search fails. Semantic search converts both into vector embeddings that capture meaning, so "stealing" and "theft" produce similar vectors and match correctly. This is critical for legal queries where users use informal language but documents use formal legal terminology.

Q6. Why did you choose all-MiniLM-L6-v2? What are its limitations?

Ans: Chosen for its balance of speed, size, and quality. At 22.7M parameters, it runs on CPU in milliseconds. Limitations: (1) Max input is 256 tokens (~300 words), so very long chunks may be truncated. (2) It's a general-purpose model, not fine-tuned on legal text, so it might miss domain-specific nuances. (3) It produces 384-dimensional vectors — larger models like all-mpnet-base-v2 produce 768 dimensions with better accuracy but are slower.

Q7. What is cosine similarity and why use it over Euclidean distance?

Ans: Cosine similarity measures the angle between two vectors, ignoring magnitude. This is important because we care about the direction (meaning) of the vector, not its length. Two texts about the same topic should be similar regardless of how long they are. Euclidean distance is affected by vector magnitude, which can be misleading for text embeddings where longer texts naturally have larger magnitudes.

Q8. What is HNSW and why does ChromaDB use it?

Ans: HNSW (Hierarchical Navigable Small World) is an approximate nearest neighbor algorithm. It builds a multi-layer graph of vectors. Finding exact nearest neighbors in high-dimensional space (384 dimensions) is extremely slow with brute force ($O(n)$). HNSW gives approximate results in $O(\log n)$ time by navigating through the graph layers from coarse to fine. The accuracy is typically 95-99% of exact search, which is more than sufficient for our use case.

C. About Chunking & Preprocessing

Q9. Why is chunking important? What happens if you skip it?

Ans: LLMs have a limited context window. If we send an entire 300-page Act as context, it either won't fit or the model will lose focus. Chunking breaks documents into small, focused pieces (typically 200-1500 chars). This also improves retrieval — searching within small, topic-specific chunks is more precise than searching within entire documents.

Q10. Why did you use section-based chunking instead of fixed-size?

Ans: Legal documents have a natural structure: Acts → Chapters → Sections. Each section is a self-contained legal provision. Fixed-size chunking (e.g., 500 chars) would cut sections in half, splitting a law like "73. Compensation for breach" across two chunks and losing its meaning. Section-based chunking preserves the legal integrity of each provision.

Q11. How do you handle sections that are very long?

Ans: If a section exceeds 3,000 characters, we sub-split it at sentence boundaries — looking for the last period or semicolon within a 1,500-character window. This ensures each sub-chunk is grammatically complete and ends at a logical point. We also add a 'part' number to the metadata so we know which piece of the section it is.

Q12. What chunk overlap strategy do you use?

Ans: We don't use chunk overlap in the traditional sense. Since our chunks are section-based (each section is complete), there's no risk of losing context at boundaries. Overlap is primarily needed for fixed-size chunking where arbitrary cuts might miss information. However, if we wanted to improve recall, we could add 50-100 character overlaps between sub-splits of large sections.

D. About the LLM & Prompt Engineering

Q13. What is prompt engineering and why is your system prompt so long?

Ans: Prompt engineering is the art of crafting instructions for an LLM to produce desired outputs. Our system prompt is detailed because it needs to handle multiple scenarios: legal questions (3-part structure with citations), casual chat (natural responses), edge cases (no relevant context), and safety rules (never hallucinate, always show disclaimer). Each rule exists because we encountered a specific failure mode during testing.

Q14. Why is temperature set to 0.2? What would happen at 1.0?

Ans: Temperature controls randomness in the LLM's output. At 0.2, the model is highly deterministic — it almost always picks the most probable next token, resulting in consistent, focused answers. At 1.0, the model would be more creative and varied, potentially producing different answers for the same question each time. For legal information, consistency and accuracy are critical, so we use a low temperature.

Q15. How do you handle multi-turn conversations?

Ans: We pass the last 6 messages (3 user-assistant exchanges) as conversation history to the LLM. These are formatted as HumanMessage and AIMessage objects via LangChain. The LLM sees the history and can understand references like "tell me more about that" or "what about the next section?". We limit to 6 messages to stay within the context window and avoid diluting the legal context.

Q16. What is hallucination in LLMs and how do you mitigate it?

Ans: Hallucination is when an LLM generates plausible-sounding but factually incorrect information. In legal contexts, this is dangerous — citing a non-existent section could mislead someone. Our mitigations: (1) RAG ensures the model works from actual documents, not memory. (2) The system prompt explicitly says "never make up information." (3) Low temperature (0.2) reduces creative outputs. (4) Source citations let users verify claims independently.

E. About Authentication & Full-Stack

Q17. How does JWT authentication work in your system?

Ans: When a user logs in with email/password, the Express server verifies the password against the bcrypt hash stored in MongoDB. If correct, it creates a JWT token containing the user's ID and signs it with a secret key. This token is sent to the frontend, which stores it in localStorage. On every subsequent API call, the frontend includes this token in the Authorization header. The server's auth middleware decodes the token, extracts the user ID, and attaches it to the request object. This is stateless authentication — the server doesn't need to track sessions.

Q18. Why bcrypt with salt factor 12?

Ans: bcrypt is a password hashing algorithm designed to be computationally expensive. The salt factor (cost factor) of 12 means bcrypt will perform $2^{12} = 4,096$ iterations of hashing. This makes brute-force attacks extremely slow — hashing one password takes ~250ms, meaning an attacker would need millions of years to crack a strong password. The salt is a random string added to each password before hashing, ensuring that two users with the same password have different hashes.

Q19. Why did you use MongoDB instead of SQL?

Ans: MongoDB's document model fits our data naturally. Chat sessions contain an array of messages, each with nested sources — this maps directly to MongoDB's nested document structure without needing complex JOIN queries. Also, the schema can evolve easily (e.g., adding new fields to messages) without database migrations. For a relational database, we would need separate tables for users, sessions, messages, and sources with foreign keys.

F. Scenario-Based & Critical Thinking

Q20. What if someone uploads a malicious PDF?

Ans: Currently, we don't accept user uploads — all PDFs are pre-loaded by us. If we added uploads: (1) validate file type and size, (2) scan with antivirus before processing, (3) run text extraction in a sandboxed environment, (4) sanitize extracted text for injection attacks. PyPDF2 can have vulnerabilities with malformed PDFs, so input validation is critical.

Q21. How would you add a new legal document to the system?

Ans: Simply place the new PDF in the `data/raw_pdfs/` directory and run the ingestion pipeline (`python ingest.py`). The pipeline will extract text, detect metadata from the filename, chunk it, generate embeddings, and add it to the ChromaDB vector store. No code changes needed — it's designed for easy extensibility.

Q22. What would you change if you had to support Hindi queries?

Ans: Three changes: (1) Use a multilingual embedding model like `paraphrase-multilingual-MiniLM-L12-v2` instead of the English-only model. (2) Ensure the chunking regex supports Devanagari characters. (3) Update the system prompt to allow Hindi responses. The LLM (Gemini) already supports Hindi, so the main bottleneck is the embedding model.

Q23. What is the weakest part of your system?

Ans: Honestly, the embedding model. `all-MiniLM-L6-v2` is a general-purpose model, not trained on legal text. Legal queries often use domain-specific terminology ("*locus standi*", "*suo motu*") that the model may not represent well. Fine-tuning the embedding model on Indian legal text would significantly improve retrieval accuracy. The second weakness is lack of hybrid search — pure semantic search can miss queries that use exact section numbers ("Section 302").

Q24. How do you evaluate the quality of your RAG pipeline?

Ans: Currently through manual testing. For a production system, I would: (1) build a test dataset of 100+ question-answer pairs with expected source sections, (2) measure retrieval accuracy (`hit@k`: does the correct section appear in the top-6?), (3) measure answer quality using LLM-as-judge (have GPT-4 rate answers for accuracy, completeness, and citation correctness), (4) track user satisfaction metrics in the chat UI.

Q25. What is the difference between RAG and fine-tuning? When would you use each?

Ans: RAG retrieves external knowledge at query time; fine-tuning bakes knowledge into the model's weights. Use RAG when: knowledge changes frequently, you need source citations, or you have limited training data. Use fine-tuning when: you need the model to learn a specific style/format, the knowledge is static, or you need faster inference (no retrieval step). In our case, RAG is ideal because laws are amended regularly and citations are mandatory.

15. Quick Revision Cheat Sheet

Use this for last-minute revision before an interview:

Project:	Legal Query System — AI-powered Indian legal chatbot using RAG
Frontend:	React.js (Vite) — Login, Signup, Dashboard, Chat pages
App Server:	Node.js + Express (port 3001) — JWT auth, MongoDB sessions
RAG Server:	Python + Flask (port 5000) — ML pipeline
Database:	MongoDB (users, chat sessions) + ChromaDB (vector embeddings)
PDF Extraction:	PyPDF2 — reads text from each PDF page
Text Cleaning:	Regex — removes gazette headers, page markers, whitespace
Metadata:	Filename parsing — extracts Act name and year automatically
Chunking:	Section-based — splits by legal sections, sub-splits large ones at sentence boundaries
Embeddings:	all-MiniLM-L6-v2 — 384-dim vectors, 22.7M params, runs on CPU
Vector DB:	ChromaDB — cosine similarity, HNSW index, persistent storage
Retrieval:	Top-6 chunks by cosine similarity
LLM:	Gemini 2.0 Flash (Vertex AI) — temp=0.2, max 2048 tokens
Filtering:	Similarity threshold 0.45 — removes irrelevant sources
Auth:	JWT tokens + bcrypt (salt 12) password hashing
Knowledge Base:	18 Indian legal documents covering constitutional, criminal, civil, family, cyber, tax, and environmental law
Key RAG benefit:	No hallucination — every answer is grounded in actual source documents

Final tip: When discussing this project in interviews, always start with the **PROBLEM** (legal info is inaccessible), then the **SOLUTION** (RAG-based chatbot), then the **IMPACT** (accurate, cited answers from 18 laws). Interviewers love the problem → solution → impact narrative.